

k-Nearest Neighbours (k-NN)

Contents

- 1-NN
- k -NN mit $k > 1$
- Implementierung am Beispiel der Palmer Penguins
- Was wird für k -NN benötigt?
- k -NN für Regression
- k -NN: Vor- und Nachteile
- Aufgaben

Lernziele

- Einen der einfachsten Algorithmen für Klassifikation und Regression kennenlernen
 - Den Einfluss von Feature-Skalierung verstehen
 - Train-Test-Splits, Modell-Training und -Evaluation implementieren
-
- Sehr einfaches Verfahren
 - Idee: **zwei Datenpunkte mit geringem Abstand sind sich besonders ähnlich** und werden der gleichen Klasse zugeordnet

- Zur Vereinfachung: 2-Klassen Problem (Binäre Klassifikation)

1-NN

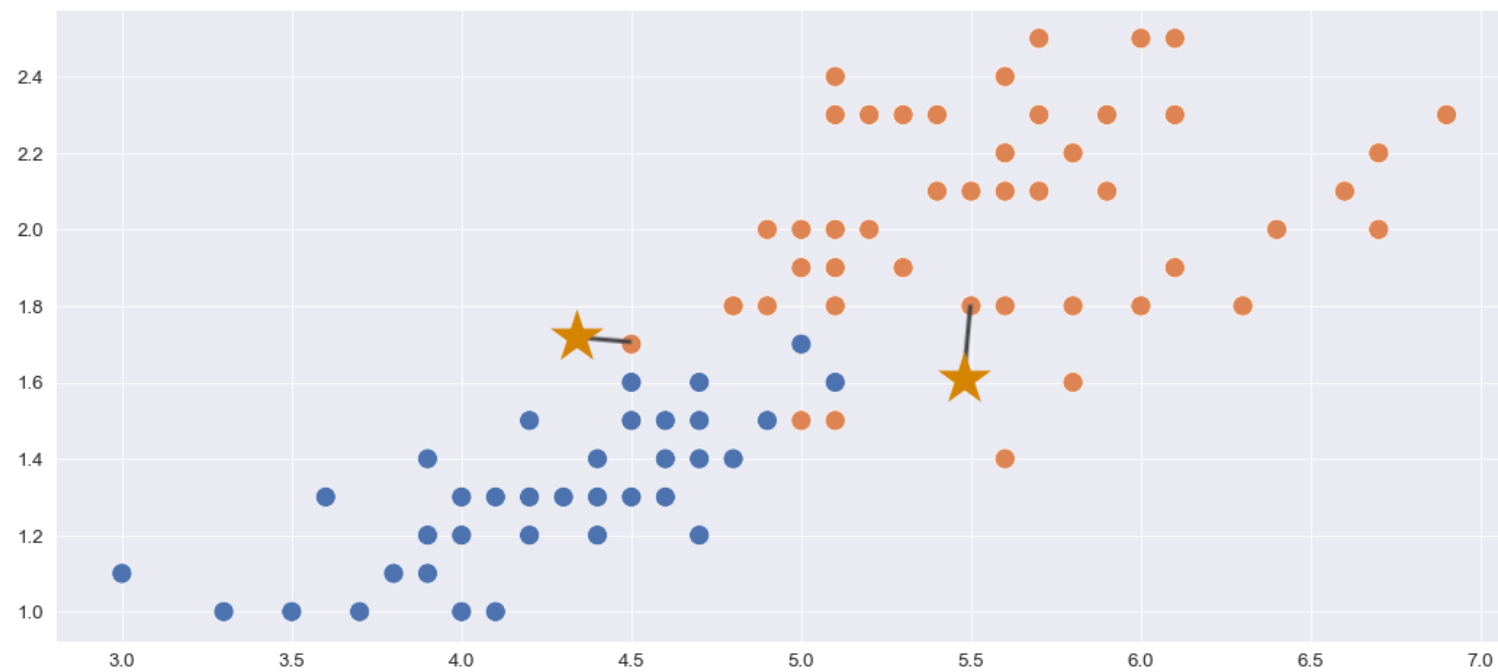
Training

Speichere die Trainingsmenge $x_1, \dots, x_m$

Anwendung auf ungesehene Daten

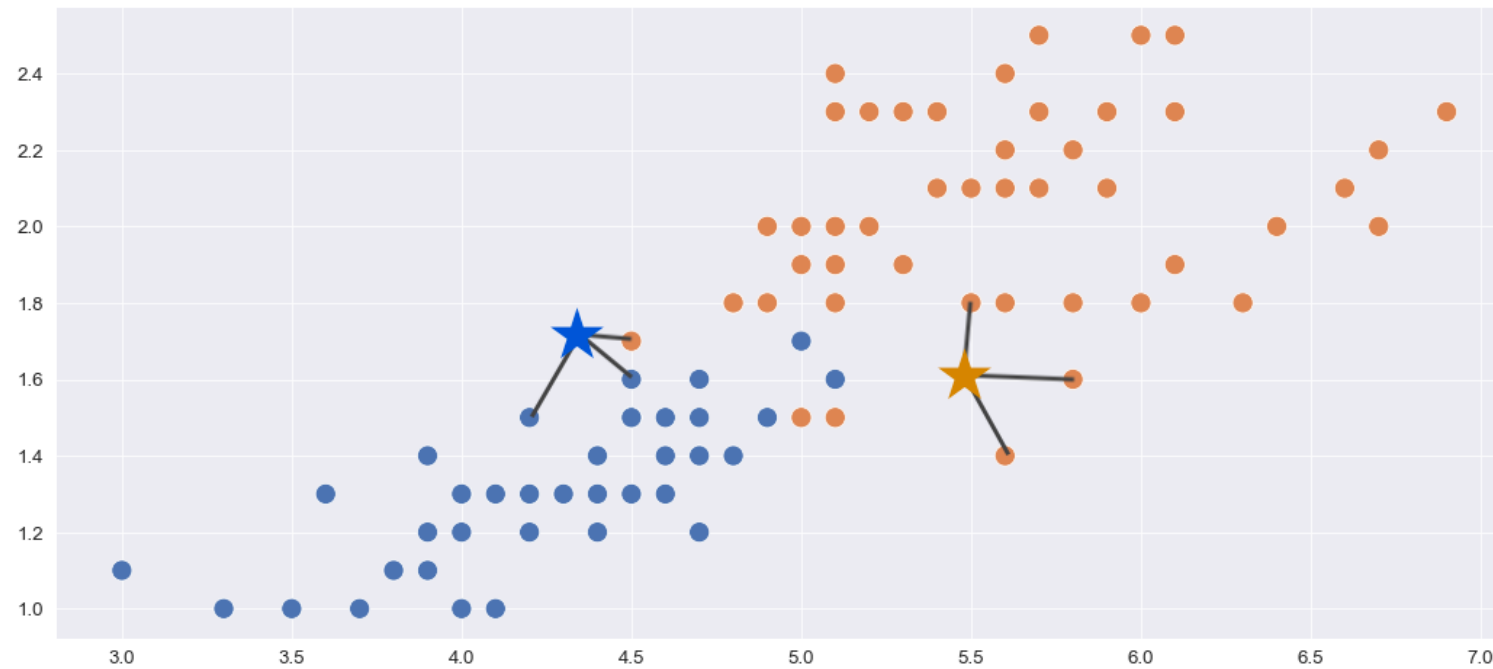
Sei x ein nicht-gelabelter Datenpunkt.

1. Berechne für alle Datenpunkte x_i in der Trainingsmenge den Abstand zu x
2. Bestimme denjenigen Datenpunkt x_i aus der Trainingsmenge, welcher den geringsten Abstand zu x hat
3. Klassifiziere x in die gleiche Klasse wie x_i



Beispiel für Klassifikation mit 1-NN

k -NN mit $k > 1$



Beispiel für Klassifikation mit 3-NN

Training

Speichere die Trainingsmenge $x_1, \dots, x_m$

Anwendung auf ungesehene Daten

Sei x ein nicht-gelabelter Datenpunkt.

1. Berechne für alle Datenpunkte x_i in der Trainingsmenge den Abstand zu x

2. Bestimme die k Datenpunkte aus der Trainingsmenge mit dem geringsten Abstand zu x
3. Klassifiziere x in diejenige Klasse, die am häufigsten unter den k nächsten Nachbarn vorkommt (*majority vote* = Modus).

Implementierung am Beispiel der Palmer Penguins

```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
from os.path import join
import sys, os

# BITTE IGNORIEREN. HIER IMPORTIERE ICH MEINE LOKALEN MODULE
sys.path.append(os.path.abspath("src"))
from ds_helpers.plotting import setup_plots, wrap_labels
setup_plots()
```

```
# ACHTUNG: Schauen Sie zuerst, wo die Datei bei Ihnen liegt und passen Sie den Pfad entsprechend an
file_path = join('data', 'penguins_lter.csv')
penguins = pd.read_csv(file_path)
penguins.sample(3)
```

	studyName	Sample Number	Species	Region	Island	Stage	Individual ID	Clutch Completion	Date Egg	Cul Le (
15	PAL0708	16	Adelie Penguin (Pygoscelis adeliae)	Anvers	Torgersen	Adult, 1 Egg Stage	N8A2	Yes	11/16/07	
21	PAL0708	22	Adelie Penguin (Pygoscelis adeliae)	Anvers	Biscoe	Adult, 1 Egg Stage	N11A2	Yes	11/12/07	
200	PAL0910	49	Chinstrap penguin (Pygoscelis antarctica)	Anvers	Dream	Adult, 1 Egg Stage	N88A1	Yes	11/23/09	

```
X_y = penguins[["Culmen Length (mm)", "Culmen Depth (mm)", "Body Mass (g)", "Species"]]
```

```
X_y.sample(5)
```

	Culmen Length (mm)	Culmen Depth (mm)	Body Mass (g)	Species
88	38.3	19.2	3950.0	Adelie Penguin (Pygoscelis adeliae)
224	47.6	14.5	5400.0	Gentoo penguin (Pygoscelis papua)
86	36.3	19.5	3800.0	Adelie Penguin (Pygoscelis adeliae)
63	41.1	18.2	4050.0	Adelie Penguin (Pygoscelis adeliae)
195	45.5	17.0	3500.0	Chinstrap penguin (Pygoscelis antarctica)

Data Preprocessing

```
X_y.dropna(inplace=True)
X_y.sample(5)
```

	Culmen Length (mm)	Culmen Depth (mm)	Body Mass (g)	Species
138	37.0	16.5	3400.0	Adelie Penguin (Pygoscelis adeliae)
89	38.9	18.8	3600.0	Adelie Penguin (Pygoscelis adeliae)
100	35.0	17.9	3725.0	Adelie Penguin (Pygoscelis adeliae)
108	38.1	17.0	3175.0	Adelie Penguin (Pygoscelis adeliae)
105	39.7	18.9	3550.0	Adelie Penguin (Pygoscelis adeliae)

```
X_y["Species"].unique(), range(len(X_y["Species"].unique()))
```

```
(array(['Adelie Penguin (Pygoscelis adeliae)',  
      'Chinstrap penguin (Pygoscelis antarctica)',  
      'Gentoo penguin (Pygoscelis papua)'], dtype=object),  
range(0, 3))
```

```
species_map = {item:index for index,item in enumerate(X_y["Species"].unique())}  
species_map
```

```
{'Adelie Penguin (Pygoscelis adeliae)': 0,  
 'Chinstrap penguin (Pygoscelis antarctica)': 1,  
 'Gentoo penguin (Pygoscelis papua)': 2}
```

```
X_y["Species"] = X_y["Species"].map(species_map)  
X_y["Species"].unique()
```

```
array([0, 1, 2])
```

Training mit `scikit-learn`

- Jedes ML model in `scikit-learn` ist als eine eigene Klasse implementiert
- k -NN Classifier wird durch die Klasse `KNeighborsClassifier` im Modul `neighbors` bereitgestellt

```
from sklearn.neighbors import KNeighborsClassifier  
from sklearn.model_selection import train_test_split
```



```
knn = KNeighborsClassifier(n_neighbors=1)
```

```
# wir ignorieren zunächst das Gewicht  
X = X_y[["Culmen Length (mm)", "Culmen Depth (mm)"]]  
y = X_y["Species"]
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, stratify=y, random_state=66)
```

```
knn.fit(X_train, y_train)
```

▼ KNeighborsClassifier ⓘ ?

```
KNeighborsClassifier(n_neighbors=1)
```

Anwendung auf neuen Datenpunkt

```
import numpy as np  
X_new = np.array([[40, 15]]) # muss ein 2-dim numpy array sein  
print(f"X_new.shape: {X_new.shape}")
```

```
X_new.shape: (1, 2)
```

```
predicted_class = knn.predict(X_new)  
print(f"Predicted output: {predicted_class}")
```

Predicted output: [2]

Evaluation auf der Testmenge

```
def inaccuracy(x_pred, y_true):  
    return np.round(sum(x_pred!=y_true)/len(y_true), 2)
```

```
X_pred = knn.predict(X_test)  
inaccuracy(X_pred, y_test)
```

0.05

```
test_data = X_test.copy()  
test_data['output'] = y_test  
test_data['pred_output'] = knn.predict(X_test)
```

```
test_data.sample(2)
```

	Culmen Length (mm)	Culmen Depth (mm)	output	pred_output
2	40.3	18.0	0	0
7	39.2	19.6	0	0

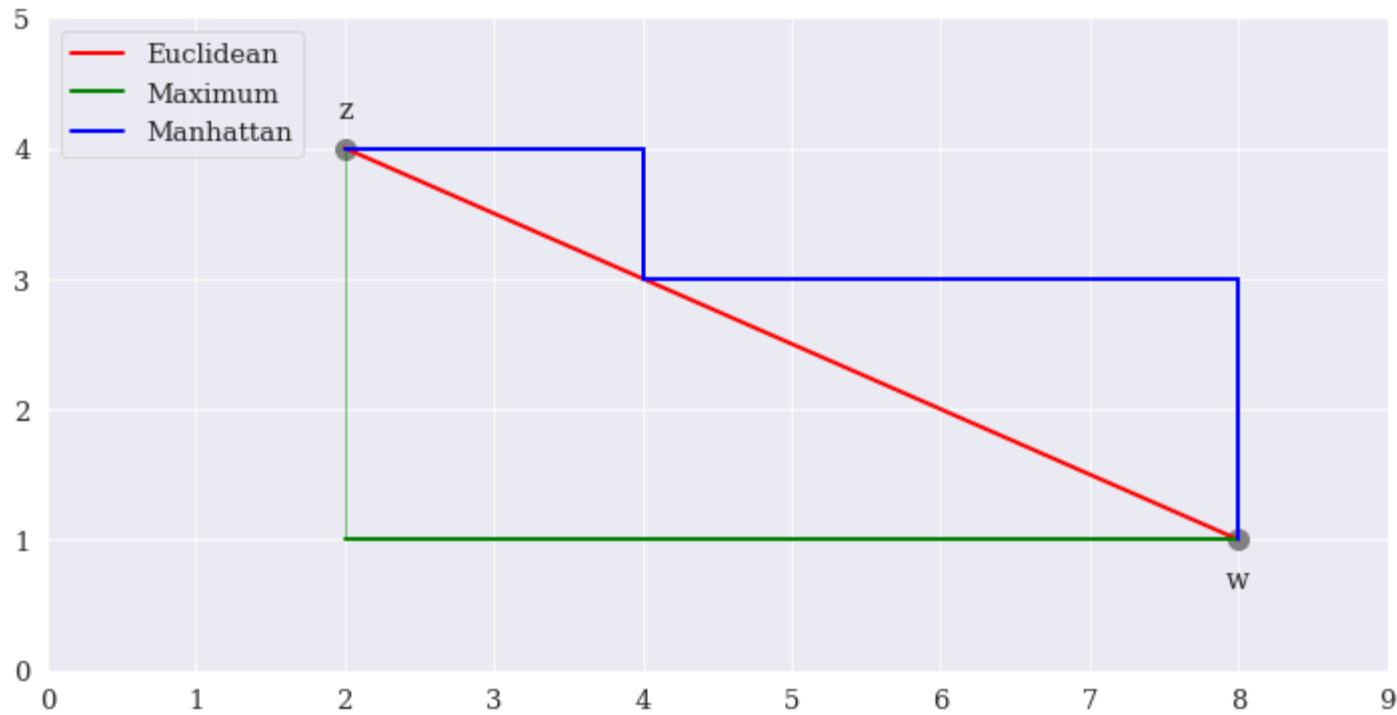
```
test_data[test_data['output']!=test_data['pred_output']]
```

	Culmen Length (mm)	Culmen Depth (mm)	output	pred_output
257	44.4	17.3	2	0
182	40.9	16.6	1	0
253	59.6	17.0	2	1
299	45.2	16.4	2	1

Was wird für k -NN benötigt?

Eine **Distanz-Funktion** d auf X :

1. $d(z, w) \geq 0$
2. $d(z, z) = 0$
3. $d(z, w) = d(w, z)$ (Symmetrie)
4. $d(z, w) \leq d(z, s) + d(s, w)$ (Dreiecksungleichung)



Drei typische Distanz-Funktionen: Euklidischer Abstand, Manhattan Abstand und Maximum-Distanz.

- **Euklidischer Abstand**

$$d_E(z, w) = \sqrt{\sum_i (z_i - w_i)^2}$$

- **Manhattan Abstand**

$$d_{\text{Man}}(z, w) = \sum_i |z_i - w_i|$$

- **Maximum-Abstand**

$$d_{\text{Max}}(z, w) = \max_i(|z_i - w_i|)$$

- Die Implementierung von k -NN in `scikit-learn` nutzt den Euklidischen Abstand per Default.
- Andere Metriken können spezifiziert/definiert werden (siehe Dokumentation)

Skalierung von Daten

```
X = X_y[["Culmen Length (mm)", "Culmen Depth (mm)", "Body Mass (g)"]]  
y = X_y["Species"]
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, stratify=y, random_state=66)
```

```
knn = KNeighborsClassifier(n_neighbors=1)  
knn.fit(X_train, y_train)
```

▼ `KNeighborsClassifier` ⓘ ?
`KNeighborsClassifier(n_neighbors=1)`

```
X_pred = knn.predict(X_test)  
inaccuracy(X_pred, y_test)
```

0.12

Der Fehler ist größer als vorher, obwohl wir ein Feature mehr haben!

```
from sklearn.preprocessing import MinMaxScaler  
scaler = MinMaxScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_train_scaled[:2]
```

```
array([[0.12851406, 0.76190476, 0.30555556],  
       [0.69879518, 0.25         , 0.63888889]])
```

```
knn = KNeighborsClassifier(n_neighbors=1)  
knn.fit(X_train_scaled, y_train)
```

▼ KNeighborsClassifier ⓘ ?

```
KNeighborsClassifier(n_neighbors=1)
```

```
X_test_scaled = scaler.transform(X_test)  
X_test_scaled_pred = knn.predict(X_test_scaled)  
inaccuracy(X_test_scaled_pred, y_test)
```

0.02

Diskussion (3-5 Min.): Durch das Skalieren der Daten, so dass jedes Feature Werte aus dem Intervall $[0,1]$ annimmt, haben wir die Ungenauigkeit des Modells deutlich reduziert. Wie erklären Sie sich das?

► **Effekt der Skalierung**

Vor der Skalierung der Features hatte `Body Mass (g)` Werte im 4-stelligen Bereich, die anderen beiden Features im 2-stelligen. Wenn man nun den Euklidischen Abstand zwischen zwei Vektoren mit jeweils diesen drei Features berechnet, so hat `Body Mass (g)` einen deutlich größeren Einfluss als die anderen beiden, die nahezu keine Rolle spielen. Durch die Skalierung haben wir allen drei Features den gleichen Einfluss auf die Distanzberechnung gegeben.

k -NN für Regression

Training

Speichere die Trainingsmenge $x_1, \dots, x_m$

Anwendung auf ungesehene Daten

Sei x ein ungesehener Datenpunkt.

1. Berechne für alle Datenpunkte x_i in der Trainingsmenge den Abstand zu x

2. Bestimme die k Datenpunkte aus der Trainingsmenge mit dem geringsten Abstand zu x
3. Sage für x den Mittelwert der y -Werte der k Datenpunkte vorher (eine andere Statistik geht auch, z.B. der Median).

```
from sklearn.neighbors import KNeighborsRegressor  
knn = KNeighborsRegressor(n_neighbors=3)
```

```
X = X_y[["Culmen Length (mm)"]]  
y = X_y["Body Mass (g)"]
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, random_state=66)
```

```
knn.fit(X_train, y_train)
```

▼ KNeighborsRegressor ⓘ ?

```
KNeighborsRegressor(n_neighbors=3)
```



```
y_pred = knn.predict(X_test)

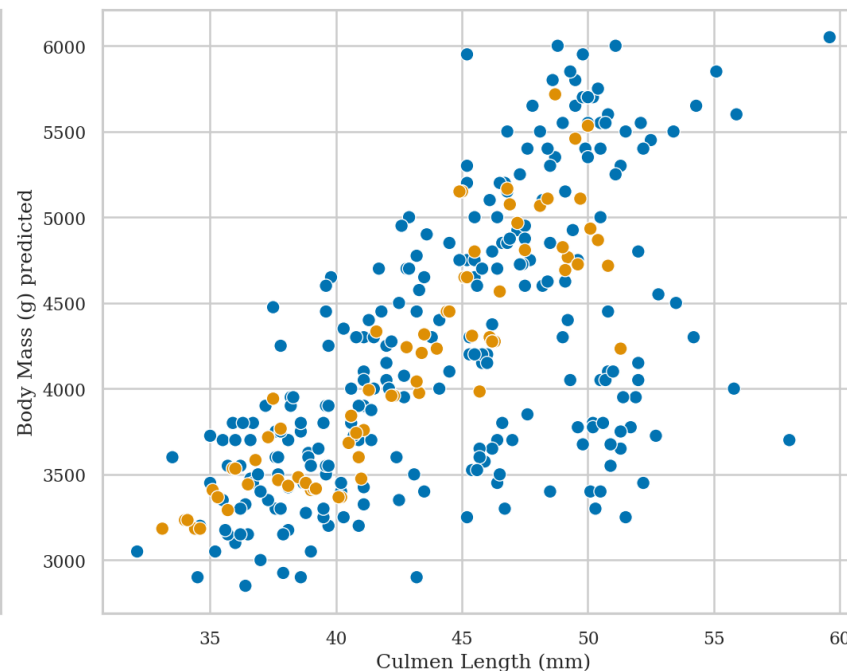
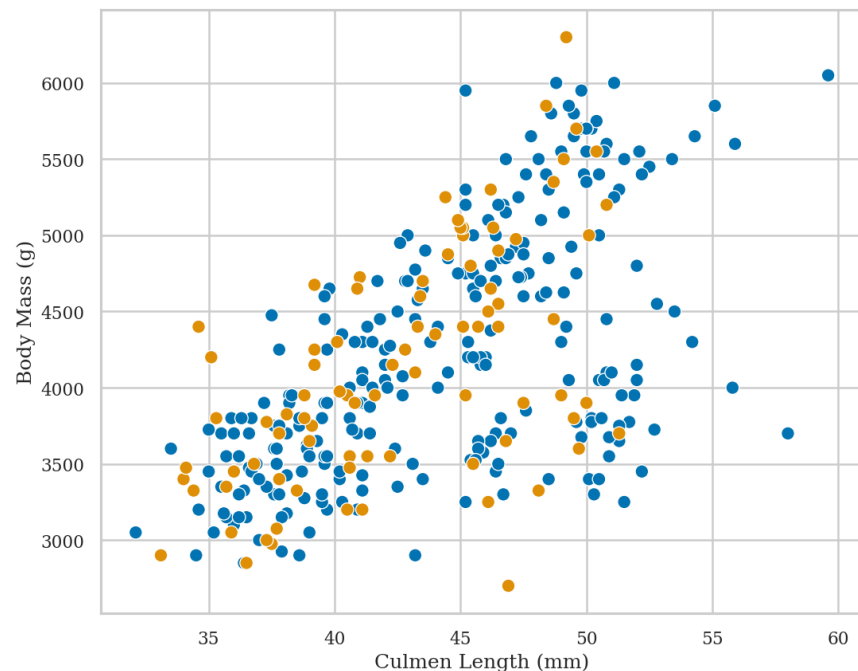
df_train = pd.DataFrame(X_train)
df_train['train/test'] = 'train'
df_train["Body Mass (g)"] = y_train
df_train["Body Mass (g) predicted"] = y_train

df_test = pd.DataFrame(X_test)
df_test['train/test'] = 'test'
df_test["Body Mass (g)"] = y_test
df_test["Body Mass (g) predicted"] = y_pred

df = pd.concat([df_train, df_test]).reset_index()
```

```
import matplotlib.pyplot as plt
import seaborn as sns # TODO: should not be required as loaded in helpers

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(18,7))
sns.scatterplot(x="Culmen Length (mm)", y="Body Mass (g)",
                data=df, hue='train/test',
                ax=ax1, color='g',
                size=[100]*len(df), sizes=(100,100), legend=False)
ax1.set(xlabel="Culmen Length (mm)", ylabel="Body Mass (g)", title='')
sns.scatterplot(x="Culmen Length (mm)", y="Body Mass (g) predicted",
                data=df, hue='train/test', ax=ax2,
                color='g', size=[100]*len(df), sizes=(100,100), legend=False)
ax2.set(xlabel="Culmen Length (mm)", ylabel="Body Mass (g) predicted", title='');
```



```
from sklearn.metrics import root_mean_squared_error
```

```
root_mean_squared_error(knn.predict(X_test), y_test)
```

751.1108268159945

- Der Wert ist etwas schwer zu interpretieren
- Wir können ihn mit einer Baseline vergleichen: Mittelwert auf y_{train} vorhersagen

```
import numpy as np
root_mean_squared_error(len(X_test)*[np.mean(y_train)], y_test)
```

776.8872615873812

- Die Vorhersage ist also kaum besser als die einfachste Baseline
- Wir werden sehen, wie wir k nutzen können, um die Performanz zu verbessern

k -NN: Vor- und Nachteile

Vorteile

- einfach zu verstehen und zu implementieren
- quasi keine Trainingszeit, entsprechend ein gutes erstes Verfahren für neue Daten
- funktioniert als Klassifikator genauso für mehrere Klassen
- macht keine Annahmen über die statistischen Eigenschaften der Daten; deshalb oft eine gute Wahl für "ungewöhnlich verteilte" Daten

Nachteile

- Rechenintensiv in der Anwendung auf ungesehene Daten, entsprechend nicht gut geeignet für produktiven Einsatz (zumindest nicht ohne Effizienz-verbessernde Maßnahmen)
- wenn eingesetzt für Klassifikation, kann eine ungleiche Verteilung der Klassen ein Problem sein, da die dominierende Klasse das *majority voting* dominiert
- nicht gut geeignet für Daten mit vielen Features (*curse of diemnsionality*)

k -NN gibt es auch in einer unüberwachten Variante; schauen wir uns vermutlich an

Aufgaben

Aufgabe 1 - Algorithmus-Logik

Gegeben ist folgender Trainingsdatensatz für einen 3-NN-Klassifikator:

```
import pandas as pd

data = pd.DataFrame([
    [-1, 1, 'A'], [0, 2, 'B'], [1, 2, 'A'], [2, 1, 'A'],
    [-1, 2, 'B'], [1, 1, 'B'], [1, 3, 'B'], [1, 0, 'B'],
    [2, 0, 'B'], [2, 2, 'A']
], columns=['x1', 'x2', 'y'])
```

- Ordnen Sie die Punkte $(3, 3)$ und $(0, 0)$ einer Klasse zu ($k = 3$).
- Was ändert sich bei $k = 5$?
- Fertigen Sie für beide Fälle Skizzen an.

Aufgabe 2 — Rechenaufwand ableiten

Angenommen, unsere Daten sind d -dimensional. Wir betrachten k -NN so, wie in der Vorlesung beschrieben, d. h.:

1. Für jeden der n Trainingspunkte den Abstand zur Query (neuer Datenpunkt) berechnen.
2. Aus allen Abständen die k kleinsten bestimmen (z. B. vollständig sortieren oder nur „top- k “ auswählen).
3. Klassifikation: Mehrheitsvotum der k Nachbarn; Regression: Mittel/Median der k Zielwerte.

- Welche Teiloperation bestimmt typischerweise den größten Rechenaufwand pro Query – Schritt 1 oder 2? Begründe.
- Wie ändert sich der Gesamtaufwand, wenn m Queries bearbeitet werden (Batch)?

Aufgabe 3 - Implementierung

Erweitern Sie eines der k -NN Modelle aus dem Unterricht, indem Sie weitere Variablen miteinbeziehen oder unterschiedliche Werte von k ausprobieren.